

CAMPUS DIGITAL FORENSICS SYSTEM

Project Report

Case ID: CASE-2026-0311

A digital forensics tool for evidence registration, chain-of-custody tracking, integrity verification, timeline reconstruction, incident analysis, and automated report generation.

Presented by:

Y.Devi Priya (192465042), T. Venkata Reddy(19265034) & S.Madhuri (192465036)

Table of Contents

1. Introduction
2. System Features
3. Source Code
4. Program Execution
5. Generated Digital Forensic Report
6. Conclusion

1. Introduction

The Campus Digital Forensics System is a Python-based command-line tool built to support a simulated digital forensic investigation, case CASE-2026-0311. It manages digital evidence from intake through analysis and final reporting, modelling the workflow a forensic investigator follows when handling compromised systems.

The system covers evidence registration and hashing, chain-of-custody logging, file integrity verification, baseline snapshot comparison, timeline reconstruction from multiple log sources, incident analysis, and automated report generation.

2. System Features

The tool is menu-driven and offers the following functions:

- Display Evidence Register — lists all registered evidence with SHA-256 hashes.
- Display Custody History — shows the chain-of-custody log for a chosen evidence item.
- Verify Integrity — recomputes an evidence file's hash and compares it to the recorded intake hash.
- Validate Files — compares a baseline hash snapshot against the current snapshot to flag new, missing, or modified files.
- Reconstruct Timeline — merges authentication, file-activity, and network logs into a single chronological timeline.
- Analyze Incident — identifies the point of compromise and lists all related events that followed it.
- Generate Report — compiles findings into a final digital forensic report.

3. Source Code

The complete Python source code is organized below by module, in the order it appears in the program.

3.1 Configuration

```
# =====  
# CONFIGURATION  
# =====  
  
DATA_DIR = "data"  
TS_FORMAT = "%Y-%m-%d %H:%M:%S"  
  
evidence_register = []  
custody_log = []
```

3.2 Utility Functions

```
# =====  
# UTILITY FUNCTIONS  
# =====  
  
def compute_sha256(file_path, chunk_size=65536):  
    sha256 = hashlib.sha256()  
  
    with open(file_path, "rb") as f:  
        while True:  
            chunk = f.read(chunk_size)  
            if not chunk:  
                break  
            sha256.update(chunk)  
  
    return sha256.hexdigest()  
  
def load_csv(filename):  
    path = os.path.join(DATA_DIR, filename)  
  
    if not os.path.exists(path):  
        return []  
  
    with open(path, newline='', encoding='utf-8') as file:  
        return list(csv.DictReader(file))
```

3.3 Evidence Registration

```
# =====  
# EVIDENCE REGISTRATION  
# =====  
  
def load_evidence():  
  
    global evidence_register
```

```

evidence_register.clear()

rows = load_csv("evidence_register.csv")

for row in rows:

    full_path = os.path.join(
os.path.dirname(__file__),
DATA_DIR,
row["file_path"].replace("./", "")
)

    if os.path.exists(full_path):
        sha256 = compute_sha256(full_path)
    else:
        sha256 = "FILE_NOT_FOUND"

    record = {
        "evidence_id": row["evidence_id"],
        "source": row["source"],
        "acquisition_datetime": row["acquisition_datetime"],
        "investigator": row["investigator"],
        "storage_location": row["storage_location"],
        "file_path": row["file_path"],
        "intake_sha256": sha256
    }

    evidence_register.append(record)

def display_evidence():

    print("\n===== EVIDENCE REGISTER =====\n")

    for record in evidence_register:

        print(f"Evidence ID      : {record['evidence_id']}")
        print(f"Source              : {record['source']}")
        print(f"Investigator         : {record['investigator']}")
        print(f"Storage Location     : {record['storage_location']}")
        print(f"SHA-256              : {record['intake_sha256']}")
        print("-" * 70)

```

3.4 Chain of Custody

```

# =====
# CHAIN OF CUSTODY
# =====

def load_custody():

    global custody_log

    custody_log.clear()

    rows = load_csv("custody_log.csv")

```

```

    for row in rows:
        custody_log.append(row)

def display_custody_history(evidence_id):

    print(f"\n===== CHAIN OF CUSTODY : {evidence_id} =====\n")

    history = [c for c in custody_log
                if c["evidence_id"] == evidence_id]

    history.sort(key=lambda x: x["datetime"])

    for item in history:

        print(item["datetime"])
        print(item["handler"])
        print(item["action"])
        print(item["notes"])
        print("-" * 60)

```

3.5 Integrity Verification

```

# =====
# INTEGRITY VERIFICATION
# =====

def verify_integrity(evidence_id):

    record = None

    for e in evidence_register:

        if e["evidence_id"] == evidence_id:
            record = e
            break

    if record is None:
        print("Evidence Not Found")
        return

    full_path = os.path.join(DATA_DIR,
                             record["file_path"].replace("./", ""))

    if not os.path.exists(full_path):
        print("Evidence File Missing")
        return

    current_hash = compute_sha256(full_path)

    print("\n===== INTEGRITY VERIFICATION =====\n")

    print("Recorded Hash:")
    print(record["intake_sha256"])

```

```

print("\nCurrent Hash:")
print(current_hash)

if current_hash == record["intake_sha256"]:
    print("\nRESULT : MATCH")
else:
    print("\nRESULT : MISMATCH")
    print("Possible Evidence Tampering")

```

3.6 Baseline Validation

```

# =====
# BASELINE VALIDATION
# =====

def validate_baseline_snapshot():

    baseline = {}
    current = {}

    for row in load_csv("baseline_hashes.csv"):
        baseline[row["filename"]] = row["baseline_sha256"]

    for row in load_csv("current_snapshot_hashes.csv"):
        current[row["filename"]] = row["current_sha256"]

    files = set(baseline.keys()) | set(current.keys())

    print("\n===== FILE VALIDATION =====\n")

    for file in sorted(files):

        if file in baseline and file in current:

            if baseline[file] == current[file]:
                status = "UNCHANGED"
            else:
                status = "MODIFIED"

        elif file in baseline:
            status = "MISSING"

        else:
            status = "NEW"

        print(f"{file:<30} {status}")

```

3.7 Timeline Reconstruction

```

# =====
# TIMELINE RECONSTRUCTION
# =====

def reconstruct_timeline():

```

```

timeline = []

for row in load_csv("auth_log.csv"):

    timeline.append({
        "timestamp": row["timestamp"],
        "source": "AUTH",
        "description":
            f"{row['event']} User={row['username']} "
            f"IP={row['source_ip']}"
    })

for row in load_csv("file_activity_log.csv"):

    timeline.append({
        "timestamp": row["timestamp"],
        "source": "FILE",
        "description":
            f"{row['action']} {row['filename']}"
    })

for row in load_csv("network_log.csv"):

    timeline.append({
        "timestamp": row["timestamp"],
        "source": "NETWORK",
        "description":
            f"Connection to {row['dest_ip']} "
            f"Bytes={row['bytes_sent']}"
    })

timeline.sort(
    key=lambda x:
        datetime.strptime(x["timestamp"], TS_FORMAT)
)

print("\n===== TIMELINE =====\n")

for event in timeline:

    print(
        event["timestamp"],
        "[", event["source"], "]",
        event["description"]
    )

return timeline

```

3.8 Incident Analysis

```

# =====
# INCIDENT ANALYSIS
# =====

def analyze_incident():

```



```

timeline = reconstruct_timeline()

print("\n===== INCIDENT ANALYSIS =====\n")

compromise = None

for event in timeline:

    if "FAILED_LOGIN" in event["description"]:
        compromise = event
        break

if compromise:

    print("POINT OF COMPROMISE")
    print(compromise["timestamp"])
    print(compromise["description"])

    print("\nRELATED EVENTS\n")

    start = datetime.strptime(
        compromise["timestamp"],
        TS_FORMAT
    )

    for event in timeline:

        current = datetime.strptime(
            event["timestamp"],
            TS_FORMAT
        )

        if current > start:
            print(event["timestamp"],
                  event["description"])

```

3.9 Report Generation

```

# =====
# REPORT GENERATION
# =====

def generate_report():

    report = []

    report.append("DIGITAL FORENSIC REPORT")
    report.append("=" * 60)
    report.append("CASE ID : CASE-2026-0311")
    report.append("")

    report.append("EVIDENCE EXAMINED")
    report.append("-" * 40)

    for e in evidence_register:
        report.append(

```

```

        f"{e['evidence_id']} - {e['source']}"
    )

    report.append("")
    report.append("FINDINGS")
    report.append("-" * 40)

    report.append(
        "1. Multiple failed logins detected."
    )

    report.append(
        "2. Successful login from external IP."
    )

    report.append(
        "3. svc_update.exe created."
    )

    report.append(
        "4. payroll_master.xlsx modified."
    )

    report.append(
        "5. finance_report_q1.pdf deleted."
    )

    report.append(
        "6. 25.22 MB transferred externally."
    )

    report.append("")
    report.append("CONCLUSION")
    report.append("-" * 40)

    report.append(
        "WKSTN-14 compromised through brute-force "
        "attack followed by file tampering and "
        "data exfiltration."
    )

    with open("report_output.txt",
              "w",
              encoding="utf-8") as file:

        file.write("\n".join(report))

    print("\nReport Generated Successfully")
    print("Saved as report_output.txt")

```

3.10 Main Menu

```

# =====
# MENU
# =====

def menu():

```

```

load_evidence()
load_custody()

while True:

    print("\n")
    print("=" * 60)
    print(" CAMPUS DIGITAL FORENSICS SYSTEM ")
    print("=" * 60)

    print("1. Display Evidence Register")
    print("2. Display Custody History")
    print("3. Verify Integrity")
    print("4. Validate Files")
    print("5. Reconstruct Timeline")
    print("6. Analyze Incident")
    print("7. Generate Report")
    print("8. Exit")

    choice = input("\nEnter Choice: ")

    if choice == "1":
        display_evidence()

    elif choice == "2":
        eid = input("Evidence ID: ")
        display_custody_history(eid)

    elif choice == "3":
        eid = input("Evidence ID: ")
        verify_integrity(eid)

    elif choice == "4":
        validate_baseline_snapshot()

    elif choice == "5":
        reconstruct_timeline()

    elif choice == "6":
        analyze_incident()

    elif choice == "7":
        generate_report()

    elif choice == "8":
        print("Exiting...")
        break

    else:
        print("Invalid Choice")

if __name__ == "__main__":
    menu()

```

4. Program Execution

The following screenshots show the program running each menu option in sequence, from the evidence register through to final report generation.

4.1 Main Menu and Evidence Register

Option 1 lists every registered evidence item together with its investigator, storage location, and intake SHA-256 hash. Note that EVD-003 and EVD-004 show FILE_NOT_FOUND, since the underlying log export files were not present at intake.

```
=====
CAMPUS DIGITAL FORENSICS SYSTEM
=====
1. Display Evidence Register
2. Display Custody History
3. Verify Integrity
4. Validate Files
5. Reconstruct Timeline
6. Analyze Incident
7. Generate Report
8. Exit

Enter Choice: 1

===== EVIDENCE REGISTER =====

Evidence ID      : EVD-001
Source           : WKSTN-14 Hard Disk Image
Investigator     : A. Kumar
Storage Location : Locker-3/Shelf-B
SHA-256          : 00238f9bd39ebdf16f9e5524bd73fe9cdd653e7e6a93915bf4a9cc9f7e86cff4
-----
Evidence ID      : EVD-002
Source           : WKSTN-14 RAM Capture
Investigator     : A. Kumar
Storage Location : Locker-3/Shelf-B
SHA-256          : 4c37cdfadef2d126749ae82c86ed98bdcd770c33f13abc40066ec619306e1933
-----
Evidence ID      : EVD-003
Source           : Domain Controller Auth Log Export
Investigator     : S. Rao
Storage Location : Locker-3/Shelf-C
SHA-256          : FILE_NOT_FOUND
-----
Evidence ID      : EVD-004
Source           : Perimeter Firewall Network Log Export
Investigator     : S. Rao
Storage Location : Locker-3/Shelf-C
SHA-256          : FILE_NOT_FOUND
```

Figure 1. Main menu and evidence register output.

4.2 Chain of Custody

Option 2 displays the custody history for a chosen evidence item (EVD-001), showing each handler, action, and timestamp from collection through examination.

```
Enter Choice: 2
Evidence ID: EVD-001

===== CHAIN OF CUSTODY : EVD-001 =====

2026-03-11 09:15:00
A. Kumar
Collected
Imaged from WKSTN-14 using write blocker
-----
2026-03-11 09:40:00
A. Kumar
Transferred to Storage
Placed in Locker-3/Shelf-B
-----
2026-03-12 14:00:00
S. Rao
Checked out for Examination
Static analysis of disk image
-----
2026-03-12 17:30:00
S. Rao
Returned to Storage
Examination complete for the day
-----

=====
CAMPUS DIGITAL FORENSICS SYSTEM
=====
1. Display Evidence Register
2. Display Custody History
3. Verify Integrity
4. Validate Files
5. Reconstruct Timeline
6. Analyze Incident
7. Generate Report
8. Exit
```

Figure 2. Chain-of-custody history for EVD-001.

4.3 Integrity Verification

Option 3 recomputes the SHA-256 hash of an evidence file and compares it against the hash recorded at intake. For EVD-001, the recorded and current hashes match, confirming the evidence has not been altered.

```

Enter Choice: 3
Evidence ID: EVD-001

===== INTEGRITY VERIFICATION =====

Recorded Hash:
00238f9bd39ebdf16f9e5524bd73fe9cdd653e7e6a93915bf4a9cc9f7e86cff4

Current Hash:
00238f9bd39ebdf16f9e5524bd73fe9cdd653e7e6a93915bf4a9cc9f7e86cff4

RESULT : MATCH

=====
CAMPUS DIGITAL FORENSICS SYSTEM
=====
1. Display Evidence Register
2. Display Custody History
3. Verify Integrity
4. Validate Files
5. Reconstruct Timeline
6. Analyze Incident
7. Generate Report
8. Exit

```

Figure 3. Integrity verification result for EVD-001 — MATCH.

4.4 File Validation

Option 4 compares a baseline hash snapshot against the current snapshot for the files on the compromised workstation, flagging each as UNCHANGED, MODIFIED, NEW, or MISSING.

```

Enter Choice: 4

===== FILE VALIDATION =====

backup_schedule.txt           UNCHANGED
finance_report_q1.pdf         MISSING
hr_policies.docx              UNCHANGED
payroll_master.xlsx           MODIFIED
svc_update.exe                NEW
system_config.ini             UNCHANGED

=====
CAMPUS DIGITAL FORENSICS SYSTEM
=====
1. Display Evidence Register
2. Display Custody History
3. Verify Integrity
4. Validate Files
5. Reconstruct Timeline
6. Analyze Incident
7. Generate Report
8. Exit

```

Figure 4. Baseline file validation results.

4.5 Timeline Reconstruction

Option 5 merges the authentication log, file-activity log, and network log into a single chronological timeline of events.

```

Enter Choice: 5

===== TIMELINE =====

2026-03-10 23:58:02 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:11 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:19 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:27 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:35 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:44 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:52 [ AUTH ] SUCCESSFUL_LOGIN User=jsmith IP=203.0.113.44
2026-03-11 00:15:30 [ AUTH ] SUCCESSFUL_LOGIN User=jsmith IP=203.0.113.44
2026-03-11 00:18:05 [ FILE ] CREATED svc_update.exe
2026-03-11 00:19:12 [ FILE ] MODIFIED payroll_master.xlsx
2026-03-11 00:20:47 [ FILE ] DELETED finance_report_q1.pdf
2026-03-11 00:21:30 [ FILE ] ACCESSED backup_schedule.txt
2026-03-11 00:22:05 [ NETWORK ] Connection to 198.51.100.77 Bytes=15820000
2026-03-11 00:23:40 [ NETWORK ] Connection to 198.51.100.77 Bytes=9400000
2026-03-11 07:45:00 [ AUTH ] SUCCESSFUL_LOGIN User=jsmith IP=10.0.4.22
2026-03-11 07:46:10 [ FILE ] ACCESSED hr_policies.docx
2026-03-11 08:05:00 [ NETWORK ] Connection to 10.0.1.5 Bytes=52000

```

Figure 5. Reconstructed timeline of authentication, file, and network events.

4.6 Incident Analysis

Option 6 rebuilds the timeline and identifies the point of compromise — the first failed login — then lists every related event that occurred afterward.

```

Enter Choice: 6

===== TIMELINE =====

2026-03-10 23:58:02 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:11 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:19 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:27 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:35 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:44 [ AUTH ] FAILED_LOGIN User=jsmith IP=203.0.113.44
2026-03-10 23:58:52 [ AUTH ] SUCCESSFUL_LOGIN User=jsmith IP=203.0.113.44
2026-03-11 00:15:30 [ AUTH ] SUCCESSFUL_LOGIN User=jsmith IP=203.0.113.44
2026-03-11 00:18:05 [ FILE ] CREATED svc_update.exe
2026-03-11 00:19:12 [ FILE ] MODIFIED payroll_master.xlsx
2026-03-11 00:20:47 [ FILE ] DELETED finance_report_q1.pdf
2026-03-11 00:21:30 [ FILE ] ACCESSED backup_schedule.txt
2026-03-11 00:22:05 [ NETWORK ] Connection to 198.51.100.77 Bytes=15820000
2026-03-11 00:23:40 [ NETWORK ] Connection to 198.51.100.77 Bytes=9400000
2026-03-11 07:45:00 [ AUTH ] SUCCESSFUL_LOGIN User=jsmith IP=10.0.4.22
2026-03-11 07:46:10 [ FILE ] ACCESSED hr_policies.docx
2026-03-11 08:05:00 [ NETWORK ] Connection to 10.0.1.5 Bytes=52000

===== INCIDENT ANALYSIS =====

```

Figure 6. Timeline output preceding the incident analysis.

```
===== INCIDENT ANALYSIS =====  
  
POINT OF COMPROMISE  
2026-03-10 23:58:02  
FAILED_LOGIN User=jsmith IP=203.0.113.44  
  
RELATED EVENTS  
  
2026-03-10 23:58:11 FAILED_LOGIN User=jsmith IP=203.0.113.44  
2026-03-10 23:58:19 FAILED_LOGIN User=jsmith IP=203.0.113.44  
2026-03-10 23:58:27 FAILED_LOGIN User=jsmith IP=203.0.113.44  
2026-03-10 23:58:35 FAILED_LOGIN User=jsmith IP=203.0.113.44  
2026-03-10 23:58:44 FAILED_LOGIN User=jsmith IP=203.0.113.44  
2026-03-10 23:58:52 SUCCESSFUL_LOGIN User=jsmith IP=203.0.113.44  
2026-03-11 00:15:30 SUCCESSFUL_LOGIN User=jsmith IP=203.0.113.44  
2026-03-11 00:18:05 CREATED svc_update.exe  
2026-03-11 00:19:12 MODIFIED payroll_master.xlsx  
2026-03-11 00:20:47 DELETED finance_report_q1.pdf  
2026-03-11 00:21:30 ACCESSED backup_schedule.txt  
2026-03-11 00:22:05 Connection to 198.51.100.77 Bytes=15820000  
2026-03-11 00:23:40 Connection to 198.51.100.77 Bytes=9400000  
2026-03-11 07:45:00 SUCCESSFUL_LOGIN User=jsmith IP=10.0.4.22  
2026-03-11 07:46:10 ACCESSED hr_policies.docx  
2026-03-11 08:05:00 Connection to 10.0.1.5 Bytes=52000
```

Figure 7. Point of compromise and related events.

4.7 Report Generation

Option 7 compiles the findings into a final report and saves it as report_output.txt.


```
=====
CAMPUS DIGITAL FORENSICS SYSTEM
=====
1. Display Evidence Register
2. Display Custody History
3. Verify Integrity
4. Validate Files
5. Reconstruct Timeline
6. Analyze Incident
7. Generate Report
8. Exit

Enter Choice: 7

Report Generated Successfully
Saved as report_output.txt

=====
CAMPUS DIGITAL FORENSICS SYSTEM
=====
1. Display Evidence Register
2. Display Custody History
3. Verify Integrity
4. Validate Files
5. Reconstruct Timeline
6. Analyze Incident
7. Generate Report
8. Exit

Enter Choice: |
```

Figure 8. Report generation confirmation and return to main menu.

5. Generated Digital Forensic Report

The report produced by the system (report_output.txt) is reproduced below.

```
DIGITAL FORENSIC REPORT
=====
CASE ID : CASE-2026-0311

EVIDENCE EXAMINED
-----
EVD-001 - WKSTN-14 Hard Disk Image
EVD-002 - WKSTN-14 RAM Capture
EVD-003 - Domain Controller Auth Log Export
EVD-004 - Perimeter Firewall Network Log Export

FINDINGS
-----
1. Multiple failed logins detected.
2. Successful login from external IP.
3. svc_update.exe created.
4. payroll_master.xlsx modified.
5. finance_report_q1.pdf deleted.
6. 25.22 MB transferred externally.

CONCLUSION
-----
WKSTN-14 compromised through brute-force attack followed by file tampering and data exfiltration.
```

Figure 9. Contents of the generated report_output.txt.

```
DIGITAL FORENSIC REPORT
=====
CASE ID : CASE-2026-0311

EVIDENCE EXAMINED
-----
EVD-001 - WKSTN-14 Hard Disk Image
EVD-002 - WKSTN-14 RAM Capture
EVD-003 - Domain Controller Auth Log Export
EVD-004 - Perimeter Firewall Network Log Export

FINDINGS
-----
1. Multiple failed logins detected.
2. Successful login from external IP.
3. svc_update.exe created.
4. payroll_master.xlsx modified.
5. finance_report_q1.pdf deleted.
6. 25.22 MB transferred externally.

CONCLUSION
-----
WKSTN-14 compromised through brute-force attack followed by file tampering and data exfiltration.
```

6. Conclusion

The evidence timeline shows a clear attack chain against WKSTN-14: a sequence of failed logins from external IP 203.0.113.44 was followed by a successful login from the same address, after which the attacker created `svc_update.exe`, modified `payroll_master.xlsx`, deleted `finance_report_q1.pdf`, and transferred approximately 25.22 MB of data to an external host (198.51.100.77).

Based on this reconstruction, WKSTN-14 was compromised through a brute-force login attack, followed by file tampering and data exfiltration. The integrity check on EVD-001 confirmed the disk image evidence remains unaltered, supporting the reliability of the findings in this report.